PHASE 2 · AGENT CODE (LOCAL)

Phase 2·Code — Loader link, route service, live map

You can show after it: Send the loader link by QR/WhatsApp → a loader without login loads the truck step by step on a phone in Hindi; routes come from the route service; all trucks on the live map.

RUN WITH	BRANCH	TIME · MODEL
Agent Code — Claude desktop app, local (Account 1)	`code/phase2-loader-map`	≈ 3 h · Sonnet

Starts after	Runs at the same time as	Builds (Architecture section 8)
Merge 1 (tag v1)	2·Git — Route planning and trip cost	F13 Loader link and loader page (no login) F14 Route service (Edge Function route-plan) F15 Live map

1. Your lane and the other agent's lane

	You — 2·Code	Other agent — 2·Git
Builds	F13 Loader link and loader page (no login) F14 Route service (Edge Function route-plan) F15 Live map	F16 Route planning and trip cost
Writes only in	`src/features/{loader,live-map}`, `supabase/functions/route-plan`, `supabase/sql/phase2code_*.sql`.	`src/features/routes`.

Never touch: `src/features/routes` (Agent Git). Need something from the other lane? Use the fallback in your build plan or write a CONTRACT CHANGE REQUEST (Engineering Reference 12.4).

2. Already built when you start

- Project, database (+ real POIs), contracts, module + slot system, router.
- Sign-in pages, accounts, demo trucks for every new company.
- Design system incl. Hindi button, `td()` demo-data translation, NextStepCard, StepTabs, BigTile, MapView.
- Shell (rail / bottom tabs, top bar with Hindi button, bell, avatar menu), Home with slots `home.actions` + `home.widgets`, Settings with slot `settings.sections`.
- Trucks (/trucks, slot `truck.tabs`), Trips (/trips, trip page with StepTabs, slots `trip.steps` + `trip.actions`), Cargo list (slot `cargo.addMethods`).
- Packing engine + 3D load plan (public API pack, fitAdditional, TruckScene, TopView, SideView, usePlanForTrip); dark map style in `public/map/`.

Read before planning: CLAUDE.md, docs/PROGRESS.md, the Plan PDF (sections 0, 1, 4 and your features in 8), Design System sections 2–6, 7.8, 7.9, 7.11, 8, 13, Engineering Reference sections 1, 3, 5, 6, 8, and the code in `src/app`, `src/components/common`, `src/types/domain.ts` and the `index.ts` of features you use.

3. Start the session

1. Open the **Claude desktop app** (Account 1) → **Code** → **Local** → folder `spareway`.
2. New session, **Plan mode**, model as shown, paste the prompt.

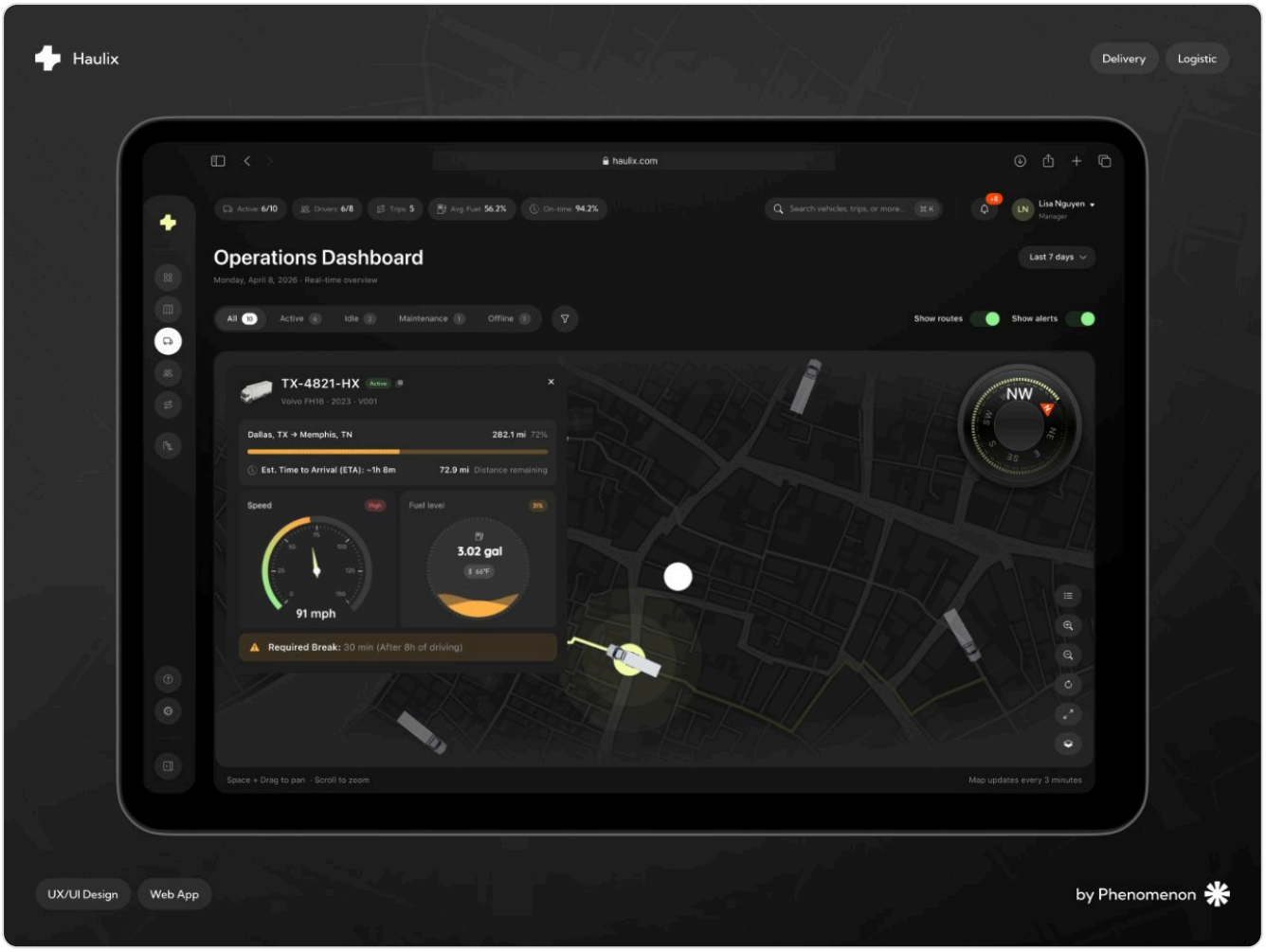
```
You are Agent Code (local, with the Supabase MCP) for Spareway. Start Phase 2·Code. Read
docs/Spareway_01_Plan_Architecture_and_Spec.pdf, docs/Spareway_02_Design_System.pdf and
docs/Spareway_03_Engineering_Reference.pdf (all pages, in parts) and
docs/phases/Phase2_Code_Loader_Link_Route_Service_and_Live_Map.pdf. Also read CLAUDE.md
and docs/PROGRESS.md if they exist. Work on branch code/phase2-loader-map (create it from
main). Ask me your questions first, then show your plan. Do not write code before I
approve the plan.
```

4. Questions Claude must ask first

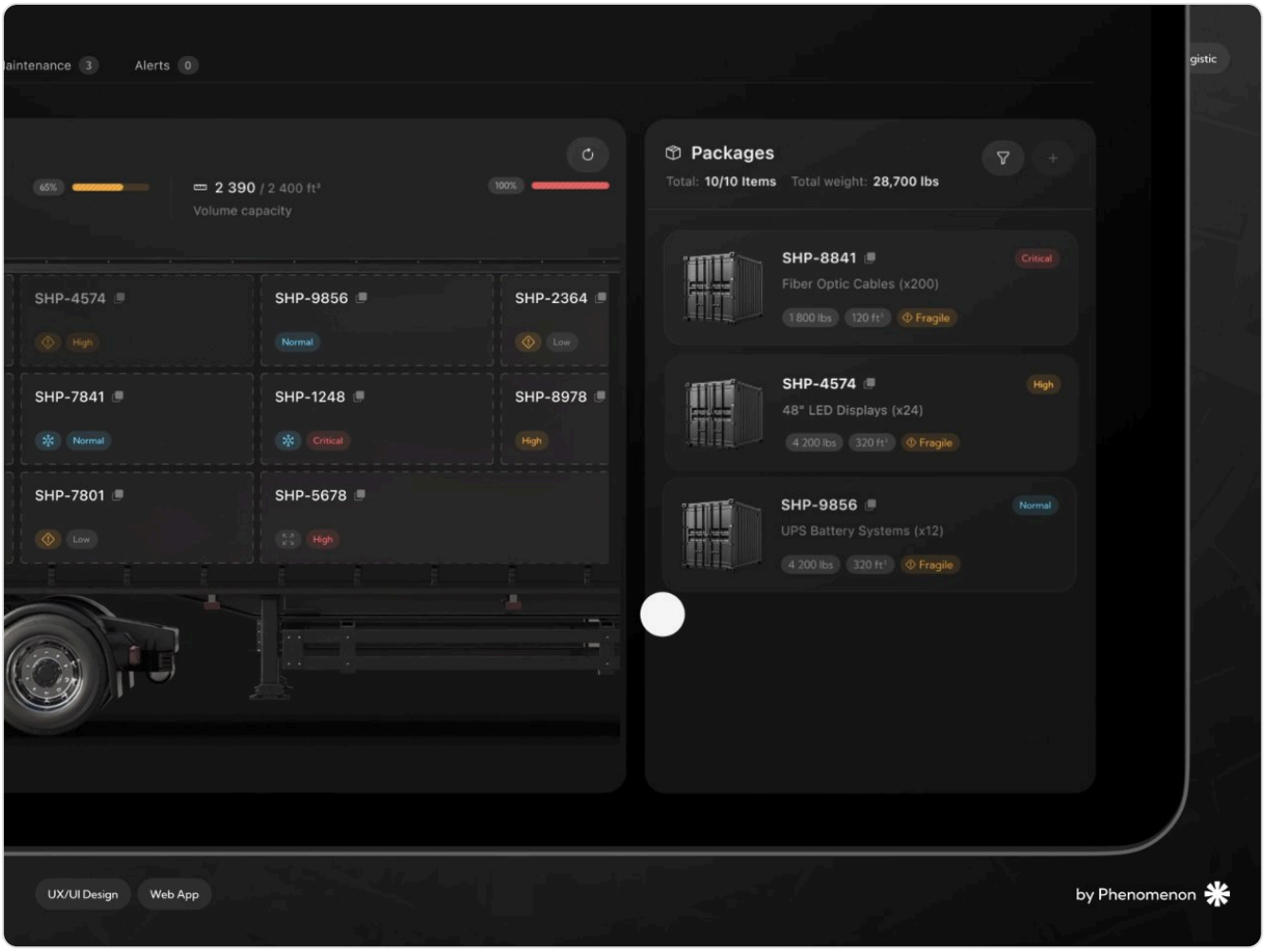
1. Please give the **OpenRouteService key** (or set Supabase secret `ORS_API_KEY` and tell me).
2. Loader page default language Hindi (default) or English?
3. Simulate truck movement on the live map with a “Simulated” badge (default yes)?

5. Build plan (in this order)

1. **route-plan first** (F14): secret, deploy with MCP, test the example, push early and tell the user “route-plan deployed”.
2. **Loader step + loader page** (F13) as Design System 7.8–7.9; test on the phone over the hotspot (https LAN address).
3. **Live map** (F15) as Design System 7.11; slots.
4. Hindi; push.



Haulix reference 6 — Live map with selected-truck card, gauges, break banner — live map: card over the map — simplified, no gauges.



Haulix reference 5 — Cargo grid + Packages list close-up — loader step card style.

6. Check before saying done

- ☐ `npm run check` and `npm run build` pass with zero errors.
- ☐ Loader link on a phone in a private tab: steps, Done, Problem → owner alert in Hindi and English; Re-plan → page continues; Turn off → "This link has ended".
- ☐ route-plan returns 3 routes for the demo trip; second call cached; no session → 401; key never in the browser.
- ☐ /map shows trucks; selected card; Simulated badge; low-data uses the lite style.
- ☐ Every new screen tested with the Hindi button (labels + demo data).
- ☐ Design System section 13 check done.

7. Hand-off

1. Update the Phase 2·Code section of `docs/PROGRESS.md` (Engineering Reference 12.2): built, how to show it, open points.
2. Add assumptions to `docs/DECISIONS.md`.
3. Push `code/phase2-loader-map` (do not merge to main).
4. Tell the user "Phase 2·Code ready to merge" + the 3 most important things to check.

8. Merge session (fresh Agent Code session, after both parts say “ready to merge”)

```
You are Agent Code. Merge Phase 2: merge origin/code/phase2-loader-map and then
origin/git/phase2-routes into main (Engineering Reference 11). Apply any
supabase/sql/phase2git_*.sql with the MCP, regenerate src/types/database.ts, run npm run
check and npm run build after each merge, then run the "Code must verify" list for Phase
2·Git in the real app (laptop + phone, English + Hindi) and fix small integration issues
(note them in DECISIONS.md). Report, then push main and tag v2.
```

- Merge `code/phase2-loader-map`, then `git/phase2-routes`; choose a route on the demo trip; loader link end-to-end on the phone; tag `v2` (mentor checkpoint 1).
- After the tag, start Phase 3 on both agents from the new main.